Detecting and Migrating Quantum-Vulnerable Cryptography in Blockchain Source Code

Benidict David
ORCID: 0009-0003-1356-7903
Amity Institute of Information Technology
Amity University Rajasthan, Jaipur, Rajasthan, India
benidictdavid18@gmail.com

Abstract

Blockchains rely on RSA and ECDSA, and a sufficiently large quantum computer breaks both. Blockchains suffer more than most systems from this, because every public key they have recorded stays public forever, so an attacker who obtains a quantum computer years from now can still attack data written today. Before an organisation can move to safer algorithms it needs a list of exactly where in its code the weak ones are used. Existing smart contract tools do not produce that list, because they look for logic defects rather than for which cryptography a program uses. This paper presents a toolkit that locates quantum-vulnerable call sites in Solidity, Python, JavaScript and Go source, gives the NIST-approved replacement for each, exports a CycloneDX cryptographic bill of materials, writes a report for regulators, and records a tamper-proof summary on a public blockchain. The main result concerns accuracy. A pattern-matching detector of this kind is easy to make find everything but hard to stop from being wrong, and two causes account for almost all of its mistakes: it matches algorithm names written in comments and strings, and it reports one line several times when several patterns match it. We eliminate both using offset-preserving masking and per-line grouping, and we replace a severity-sum risk score that assigned nearly every file the same value. On a hand-labelled benchmark of seven files containing nineteen marked call sites, three of which are files written specifically to mislead the detector, these changes raise precision from 0.633 to 1.000 while recall remains at 1.000. Because the benchmark was written by the same author as the detector, the paper states explicitly what the result does and does not establish, and reports the Wilson confidence interval that bounds it. The detection logic is additionally implemented in a second runtime that executes entirely in a web browser and is verified to produce identical output, so proprietary source never has to be transmitted.

Keywords: Post-quantum cryptography; Blockchain security; Static analysis; Cryptographic bill of materials; ML-DSA; ML-KEM; Smart contract analysis; Regulatory compliance

1. Introduction

Blockchains depend on a small number of cryptographic algorithms. Ethereum and Bitcoin sign transactions using ECDSA on the secp256k1 curve. Smart contracts verify signatures through the `ecrecover` function. The servers and scripts surrounding a deployment typically use RSA for key exchange and SHA-256 for hashing. Every one of these rests on a mathematical problem that a quantum computer solves efficiently. Shor's algorithm [1] factors large numbers and solves discrete logarithms in polynomial time, which breaks RSA and ECDSA completely rather than merely weakening them. Grover's algorithm [2] is a milder threat that speeds up brute-force search, halving the effective strength of hash functions and block ciphers.

Blockchains face a difficulty here that most systems do not. Normally, once a weak algorithm is retired, the data that passed through it is no longer available to an attacker. A public ledger inverts this: every public key and signature it has ever stored remains permanently retrievable [3]. An attacker who acquires quantum capability in a decade can therefore still attack the entire history. Mosca's rule [4] states the consequence plainly: when the time data must remain secret, plus the time migration takes, exceeds the time until quantum computers arrive, the organisation is already late. For an immutable ledger the secrecy requirement is unbounded, so the work should already have started.

1.1 Problem Statement

Given a source file drawn from a blockchain project, produce the list of exact locations at which quantum-vulnerable cryptography is invoked, such that the list is specific enough to plan a migration and accurate enough that a developer reading it continues to trust the tool. Accuracy is the substantive difficulty. A pattern-matching detector is straightforward to make complete, in the sense of finding every genuine call site, and surprisingly hard to make precise, because algorithm names occur heavily in comments and string literals and

because overlapping patterns report a single call site more than once. Both faults inflate every downstream figure, including the risk score and the remediation estimate.

1.2 Objectives

1. Build a detector covering five families of quantum-vulnerable primitives across Solidity, Python, JavaScript and Go source.
2. Identify the dominant sources of false positives in such a detector and eliminate them without reducing recall.
3. Design a risk score that discriminates between source files rather than saturating at its maximum.
4. Construct a hand-labelled benchmark that includes deliberate hard negatives, and release it with its labelling policy.
5. Deliver outputs an organisation can act on: a standards-conformant cryptographic inventory, regulator-facing reports, and a verifiable record.

1.3 Research Questions

- **RQ1.** What causes false positives in pattern-based detection of quantum-vulnerable cryptography in blockchain source code, and how much does each cause contribute?
- **RQ2.** Can those causes be eliminated without any loss of recall?
- **RQ3.** Does severity-sum risk scoring discriminate between source files, and does a multi-factor score improve on it?
- **RQ4.** Can the detection logic be ported to a client-side runtime without behavioural divergence from the reference implementation?

1.4 Brief Description of the Work

The system processes a source file through five stages: comments and string literals are blanked while preserving byte offsets, the remaining executable text is matched against patterns for five algorithm families, overlapping matches are grouped per line and family, the surviving findings are reduced to a single exposure score, and a migration plan is generated mapping each family to its NIST replacement. Three documents are derived from these results: a CycloneDX 1.6 cryptographic bill of materials, a report rendered against a regulator-specific template, and a Keccak-256 digest recorded on the Ethereum Sepolia network. The detector is implemented twice, in Python and in JavaScript, and the two are verified to agree.

1.5 Main Contributions

1. **Two mechanisms that make pattern-based crypto detection accurate.** Offset-preserving masking blanks comments and strings by substituting spaces rather than deleting characters, so reported line numbers stay exact; per-line grouping collapses overlapping matches so one call site yields one finding.
2. **A non-saturating exposure score.** The severity-sum score used in the preliminary system reaches its cap at four high-severity findings and produced a mean of 99.5 across the 19 scan records gathered during development. It is replaced by a three-term model over worst severity, family breadth and finding density.
3. **A labelled benchmark with hard negatives.** Seven files and nineteen marked call sites, of which three files carry zero labels: one already post-quantum, one naming the old algorithms only in prose, and one containing identifiers that merely resemble cryptographic names.
4. **An ablation isolating each mechanism,** including the negative result that word-boundary anchoring alone removes no false positive, and an explicit account of why the absolute scores overstate field accuracy.
5. **A verified dual-runtime implementation,** agreeing on all seven benchmark files across six compared fields, which permits assessment of proprietary source without transmitting it.

1.6 Organisation of the Remaining Sections

Section 2 reviews existing smart contract and cryptographic analysis tools and identifies the research gap. Section 3 describes the dataset, the system architecture, the detection algorithm, the scoring equations and the experimental setup. Section 4 reports evaluation metrics, the ablation results, the scoring comparison and the comparison against baselines. Section 5 answers the four research questions and relates the findings to previous studies. Section 6 summarises the work, states its limitations and outlines future scope.

2. Related Work

2.1 Literature Review

Research relevant to this problem falls into three groups: analysis of smart contracts, detection of cryptographic misuse, and standardisation of post-quantum algorithms and cryptographic inventories.

Smart contract analysis. Slither [5] performs static analysis over an intermediate representation of Solidity and detects a broad catalogue of vulnerability patterns including reentrancy, unchecked arithmetic and access-control defects. Mythril applies symbolic execution to EVM bytecode, and Manticore supports symbolic exploration of both source and bytecode. These tools are mature and widely deployed, but none reports which cryptographic algorithms a contract composes, because a correctly written ECDSA verification is not a defect under their threat models.

Cryptographic misuse detection. CryptoGuard [6] identifies misuse patterns such as hard-coded keys, predictable initialisation vectors and insecure cipher modes in large Java codebases, reporting low false-positive rates. CrySL [7] provides a specification language in which the correct usage protocol of a cryptographic API can be expressed and then checked statically. Both target incorrect use of algorithms that are themselves sound. The problem addressed here is the complement: correct use of algorithms that will become unsound. A misuse detector reports nothing for a textbook-correct ECDSA implementation, which is precisely the code that must be found.

Standards and inventories. NIST finalised three post-quantum standards in August 2024: FIPS 203 for ML-KEM [8], FIPS 204 for ML-DSA [9] and FIPS 205 for SLH-DSA [10], with transition guidance setting deprecation dates [11]. Bernstein and Lange [12] survey the underlying mathematics. Webber et al. [13] estimate the hardware required to attack ECDSA in practice. On the inventory side, CycloneDX [14] introduced a cryptographic-asset component type, giving machine-readable form to the notion of a cryptographic bill of materials, and NIST SP 800-57 [15] provides the key-management framing that such an inventory supports.

Work	Primary focus	Reports crypto inventory	PQ migration guidance	Blockchain aware	Verifiable record
Slither [5]	Solidity logic defects	No	No	Yes	No
Mythril	EVM symbolic execution	No	No	Yes	No
Manticore	Symbolic exploration	No	No	Yes	No
CryptoGuard [6]	Java crypto misuse	Partial	No	No	No
CrySL [7]	Crypto API usage rules	Partial	No	No	No
CycloneDX [14]	Inventory format only	Format	No	No	No
FIPS 203/204/205 [8], [9], [10]	Algorithm standards	No	Target	No	No
This work	Quantum exposure in blockchain source	Yes	Yes	Yes	Yes

Table 1. Summary of related work against the four capabilities this problem requires. No existing tool provides all four.

2.2 Research Gap

Table 1 makes the gap explicit. Smart contract analysers are blockchain aware but report no cryptographic inventory. Cryptographic analysers report on cryptography but target misuse in non-blockchain languages and offer no migration guidance. The standards define what to migrate to but not how to find what needs migrating. CycloneDX supplies a format but no detector. No published tool combines quantum-vulnerability detection for ledger code, standards-aligned migration planning, machine-readable inventory export and a verifiable assessment record.

A second and less obvious gap concerns accuracy. Pattern-based cryptographic detection is widely used but its characteristic failure modes have not, to our knowledge, been isolated and measured. The literature on misuse detection reports aggregate false-positive rates without attributing them to specific causes. This paper addresses that gap directly, and the mechanisms it contributes apply to any tool that searches source text for cryptography.

3. Methodology

3.1 Dataset

The evaluation dataset is a benchmark of seven source files in which every active call site of a quantum-vulnerable primitive has been marked by hand. Each label records a line number and an algorithm family, and a prediction counts as a true positive only when both components match. This criterion is deliberately strict: naming the correct family on the wrong line does not tell a developer where to work.

The labelling policy was fixed before labelling began and is released with the data. Import statements are labelled, since importing a vulnerable primitive forms part of the migration surface. Call sites are labelled. A function definition is labelled when its body implements the primitive. Occurrences inside comments, documentation strings and string literals are not labelled, and neither are identifiers that merely contain an algorithm name as a substring.

Four files are positive cases: a Solidity consent registry using `ecrecover` and `keccak256`; a Python settlement signer combining ECDSA, RSA, SHA-256, short-key AES and a Diffie-Hellman exchange; a Python key service; and a partially migrated Solidity contract in which an ML-DSA path has been added while the legacy `ecrecover` path remains live. The last of these tests that a half-migrated file still reports its residual classical exposure, which is the common real-world state.

The remaining three files carry zero labels and form the hard-negative component. The first is fully post-quantum code importing `dilithium2` and `kyber512` and hashing with SHA-384. The second documents a completed migration in prose, naming `RSA.generate`, `hashlib.sha256`, `ECDSA.sign`, Diffie-Hellman and `ecrecover` across comments, a documentation string and a literal, while invoking none of them. The third contains identifiers such as `bandwidth`, `sha256_cache_enabled` and `rsa_config_path`, none of which is cryptographic. Each negative targets one of the three error classes analysed in Section 3.5. The benchmark totals nineteen labelled call sites across 347 lines of source.

3.2 Threat Model and Scope

The adversary assumed throughout follows a harvest-now, decrypt-later strategy. They record public blockchain data continuously in the present and acquire cryptographically relevant quantum capability at an unknown future point, after which they can recover private keys from the public keys already recorded and forge signatures. This adversary never needs access to the organisation's systems, because the material required is already public, which is what distinguishes the blockchain case from a conventional one.

The scope of the tool is correspondingly narrow and is stated explicitly because it bounds every result reported later. The tool answers one question: at which source locations does this file invoke a primitive that the above adversary defeats? It does not evaluate protocol design, key management, randomness quality, or whether a correctly named primitive is used correctly. The output is a migration worklist rather than a cryptographic audit, and the generated reports carry that statement in the document itself.

Four design goals follow. First, accuracy is prioritised over volume: a developer must be able to trust that every reported location requires work, because a worklist that flags documentation is discarded at first review. Second, every finding must carry an exact line, a named replacement and the standard governing it. Third, assessing proprietary code must not require transmitting it, since the organisations with the greatest exposure are the least willing to upload source to a third party. Fourth, an assessment shown to a regulator or insurer must be independently checkable for integrity and date without trusting the assessing party.

3.3 Detection Scope: Algorithm Families

Five families are covered, listed in Table 2 with the severity assigned to each and its standardised replacement. A family is rated HIGH when the adversary defeats it outright and MEDIUM when the primitive is weakened but survives with larger parameters. Each pattern targets an invocation or import form rather than a bare algorithm name, which is the first and weakest of the accuracy measures; the two mechanisms in Section 3.5 do the substantive work.

Family	Severity	Replacement	Standard	Representative pattern
ECDSA	HIGH	ML-DSA (Dilithium)	FIPS 204	ecrecover(, secp256k1
RSA	HIGH	ML-KEM (Kyber)	FIPS 203	RSA., Crypto.PublicKey
Diffie-Hellman	HIGH	ML-KEM (Kyber)	FIPS 203	diffie_hellman, ECDH
SHA-256	MEDIUM	SHA-384 / SHA-512	FIPS 180-4	hashlib.sha256(, keccak256(
AES (short key)	MEDIUM	AES-256-GCM	FIPS 197	AES.new(..MODE_, AES-128

Table 2. The five algorithm families covered, with severity, standardised replacement and an example of the patterns matched.

Two constraints shape the replacements offered for on-chain code. ML-DSA signatures and public keys are substantially larger than their ECDSA counterparts, and lattice verification is not expressible within current Ethereum gas limits at acceptable cost, so the recommendation for a contract verifying signatures on-chain is an architectural change rather than a substitution. Separately, because Ethereum address derivation depends on Keccak-256, a deployed contract cannot change its hash function without breaking every address derived from it, so hash recommendations are scoped to off-chain code.

3.4 Proposed Approach and Architecture

The system is a five-stage pipeline in which each stage is independent and communicates through JSON, so any stage can be replaced without disturbing the others. Figure 1 shows the arrangement.

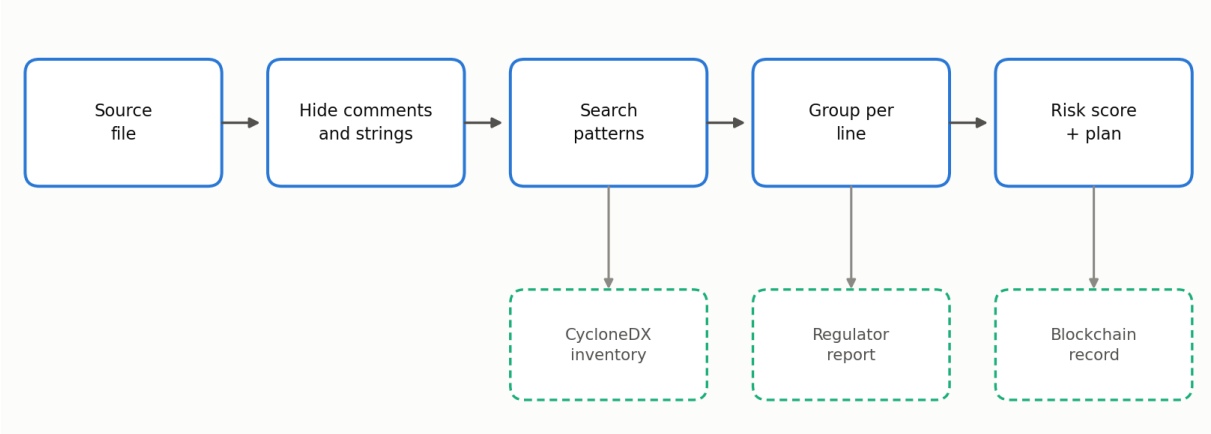


Fig. 1. System architecture. Solid boxes are the five analysis stages; dashed boxes are the governance documents derived from the stage above them.

Stages one to three constitute detection and are specified as Algorithm 1. Stage four groups findings by family and emits, for each, the replacement algorithm, its governing standard, the affected line numbers and a language-appropriate code template. For Solidity the template is the off-chain verification pattern rather than a direct substitution, because ML-DSA signatures and keys are substantially larger than their ECDSA equivalents and lattice verification cannot be executed within current Ethereum gas limits at acceptable cost. Stage five derives the three governance documents. Only a Keccak-256 digest is published on-chain; no source material is disclosed. The transaction is signed by the wallet of the party performing the assessment, so the service holds no private key.

3.5 Algorithm

Algorithm 1 states the detection procedure. The two mechanisms this paper contributes are line 2 and lines 9 to 14. Line 2 blanks non-code regions by substituting spaces rather than deleting characters, which preserves both the total length and the line structure of the buffer, so an offset in the masked copy indexes the same position in the original file and reported line numbers remain exact. Lines 9 to 14 collapse matches sharing a family and a line, retaining the longest matched text as the most specific description of the call site. Family is retained in the grouping key, so a line genuinely invoking two families still yields two findings, because it requires two separate remediations.

```
Algorithm 1: Precision-oriented detection of quantum-vulnerable cryptography
-----------------------------------------------------------------------------
Input : source text T; pattern set P grouped by family; severity map sigma
Output: findings V; informational mentions M; exposure score S
```

```
 1  L  <- split T into lines
 2  C  <- MASK(T)        // replace each char inside a comment or string
                         // literal with a space; preserve all newlines
 3  R  <- empty list     // raw matches
 4  for each family a in P do
 5      for each pattern p in P[a] do
 6          for each match m of p in C do
 7              append (a, LINE(C, m.start), m.text) to R
 8
 9  B  <- empty map      // best match per (family, line)
10  for each (a, n, text) in R do
11      if (a,n) not in B or |text| > |B[a,n].text| then
12          B[a,n] <- (a, n, text)
13
14  V  <- sort values of B by (line, family)
15
16  M  <- empty list     // names occurring ONLY in masked regions
17  for each family a in P, pattern p in P[a], match m of p in T do
18      if C[m.start .. m.end] is all whitespace then
19          append (a, LINE(T, m.start)) to M
20
21  S  <- SCORE(V, |L|)  // Equation (1)
22  return V, M, S
-------------------------------------------------------------------------
```

A worked example makes both mechanisms concrete. Consider the two-line fragment below, taken from a file mid-migration. Line 12 is documentation describing work already completed; line 13 is a live call site.

```
ORIGINAL SOURCE
  12 |  # replaced RSA.generate with ML-KEM; see migration note
  13 |  digest = hashlib.sha256(payload).digest()
AFTER MASKING (line length and newlines preserved)
  12 |
  13 |  digest = hashlib.sha256(payload).digest()
RAW MATCHES AGAINST THE MASKED BUFFER
  (SHA256, line 13, 'hashlib.sha256(')     <- qualified pattern
  (SHA256, line 13, 'sha256(')             <- bare pattern
AFTER PER-LINE GROUPING
  (SHA256, line 13, 'hashlib.sha256(')     <- longest match retained
REPORTED SEPARATELY, NOT COUNTED
  (RSA, line 12)  mentioned in a comment, not a call site
```

A detector searching the raw text reports two findings here, one of them the comment on line 12, and counts line 13 twice. The masked buffer removes the first error while leaving line 13 at line 13, so the reported location stays correct; grouping removes the second. The RSA mention is surfaced in the informational list rather than discarded, so a reviewer can confirm the tool saw it and chose not to count it.

3.6 Mathematical Formulation

The preliminary system scored a file by summing severity weights, 30 points per high-severity finding, capped at 100. Four high-severity findings reach 120, so any file containing four or more scores exactly 100 regardless of its size, the breadth of algorithms involved or their density. Across the 19 scan records accumulated during development the mean was 99.5 with negligible variance. A score that assigns nearly every input the same value carries no information, and since this figure is printed on compliance documentation the defect is consequential. It is replaced by Equation (1).

$$S = \min\Big(100,\ \max_{f \in F} \sigma(f) + \min\big(20,\ 5(|A| - 1)\big) + 20\big(1 - e^{-\rho/5}\big)\Big)$$

Equation (1). The three-term exposure score.

Here F is the set of findings and sigma maps a severity level to base points, with HIGH mapping to 60, MEDIUM to 35 and LOW to 15. A is the set of distinct algorithm families present, so the breadth term contributes 5 points per additional family up to 20. The density term uses rho = 100|F| / L, the number of findings per hundred lines in a file of L lines, and saturates smoothly so that a densely affected file outranks a sparse one without a single

pathological file dominating the scale. The score is monotonic in all three inputs and reaches 100 only for a file that is both broadly and densely affected.

Detection quality is measured by the standard quantities in Equations (2) to (4), where a prediction is correct only if both its line number and its family match a label:

```
Precision = TP / (TP + FP)      (2)
Recall    = TP / (TP + FN)      (3)
F1        = 2 · Precision · Recall / (Precision + Recall)      (4)
```

Because the benchmark is small, point estimates alone would be misleading, so Wilson score intervals are reported alongside them. For k correct out of n predictions at the 95% level the interval is centred at $(p + z^2/2n) / (1 + z^2/n)$ with $p = k/n$ and $z = 1.96$.

3.7 Experimental Setup

The reference implementation is Python 3.13 exposed through a FastAPI service of twelve endpoints. The second implementation is JavaScript executing in the browser, with no server component. Reports are produced with ReportLab in the Python runtime and jsPDF in the browser runtime. Blockchain records are written to the Ethereum Sepolia test network through a registry contract exposing a single `storeProof` function.

Three detector configurations are evaluated in order to attribute the improvement to specific mechanisms. **v1-baseline** is the original pattern set matched against raw text with no grouping; this is a genuine earlier configuration of the system, retained verbatim rather than reconstructed. **v2-boundaries** is the revised, word-anchored pattern set, still matched against raw text without grouping. **v3-full** is the revised pattern set with masking and per-line grouping, and is the shipped configuration. Runtime equivalence is checked by executing the JavaScript detection module outside the browser and comparing six fields against the Python implementation for every benchmark file: the finding set as (line, family) pairs, the finding count, the informational-mention set, the post-quantum algorithms detected, the exposure score and the line count. The full evaluation is reproducible by running the released harness.

4. Results

4.1 Evaluation Metrics

Detection is assessed by precision, recall and F1 as defined in Equations (2) to (4), computed overall and per algorithm family, with Wilson 95% intervals. Two auxiliary quantities are also reported. *Raw hits* is the total number of pattern matches before grouping, which exposes the duplicate-counting behaviour that precision alone conceals. The *exposure score* is evaluated not for accuracy, which would require expert rankings, but for discrimination: whether it assigns distinct values to files of differing severity.

4.2 Performance Analysis

Version	TP	FP	FN	Precision	Recall	F1	Raw hits
v1-baseline	19	11	0	0.633	1.000	0.776	39
v2-boundaries	19	11	0	0.633	1.000	0.776	38
v3-full	19	0	0	1.000	1.000	1.000	26

Table 3. Ablation across three detector configurations. Seven files, nineteen labelled call sites.

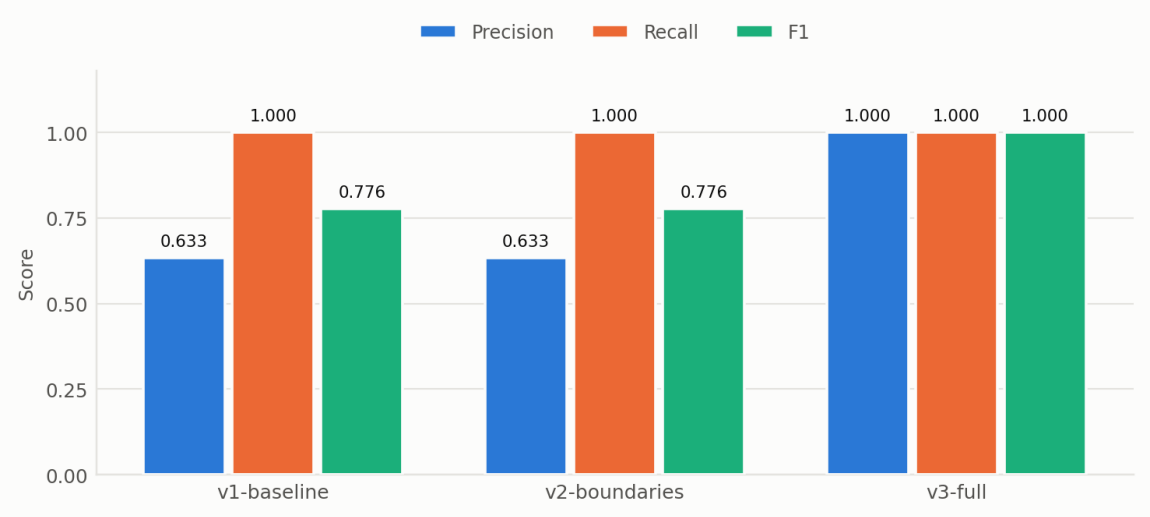


Fig. 2. Precision, recall and F1 by configuration. Recall is unchanged throughout; precision rises only when masking and grouping are both applied.

Three observations follow. First, recall remains 1.000 in every configuration: the accuracy mechanisms never suppress a genuine call site, which is the property that must hold for them to be acceptable, since precision bought with recall would be a poor trade in a security inventory.

Second, word-boundary anchoring alone yields no improvement. The v2 configuration removes one raw match relative to v1 but eliminates no false positive. This negative result is worth reporting: the substring-identifier error class that anchoring targets is real, and one benchmark file was written specifically to exercise it, but on this data the dominant error source is comment and string matching, which anchoring does not address. Anchoring is the intuitive first remedy and is insufficient alone.

Third, masking combined with per-line grouping eliminates all eleven false positives, and reduces raw hits from 38 to 26. The residual difference between 26 and the nineteen labelled call sites is accounted for entirely by the hard-negative files, where the correct count is zero. Table 4 confirms the result is uniform across families rather than driven by one of them.

Family	TP	FP	FN	Precision	Recall	F1	Precision 95% CI
ECDSA	4	0	0	1.000	1.000	1.000	[0.510, 1.000]
RSA	4	0	0	1.000	1.000	1.000	[0.510, 1.000]
SHA-256	8	0	0	1.000	1.000	1.000	[0.676, 1.000]
Diffie-Hellman	2	0	0	1.000	1.000	1.000	[0.342, 1.000]
AES (short key)	1	0	0	1.000	1.000	1.000	[0.207, 1.000]

Table 4. Per-family results for the shipped configuration (v3-full), with Wilson 95% intervals on precision. The intervals are wide because each family contributes few call sites; the single AES observation carries essentially no information.

4.3 Exposure Score Discrimination

File	Found	Families	Lines	Old score	New score
Python settlement signer	9	4	108	100.0	91.2
Python key service	5	4	26	100.0	94.6
Solidity consent registry	3	2	81	60.0	75.5
Half-migrated contract	2	2	32	45.0	79.3
Post-quantum module	0	0	39	0.0	0.0
Prose-only module	0	0	23	0.0	0.0
Trick-names module	0	0	38	0.0	0.0

Table 5. Exposure scores for every benchmark file under both scoring models. The severity-sum model ties the two most affected files at the cap; the three-term model separates all four.

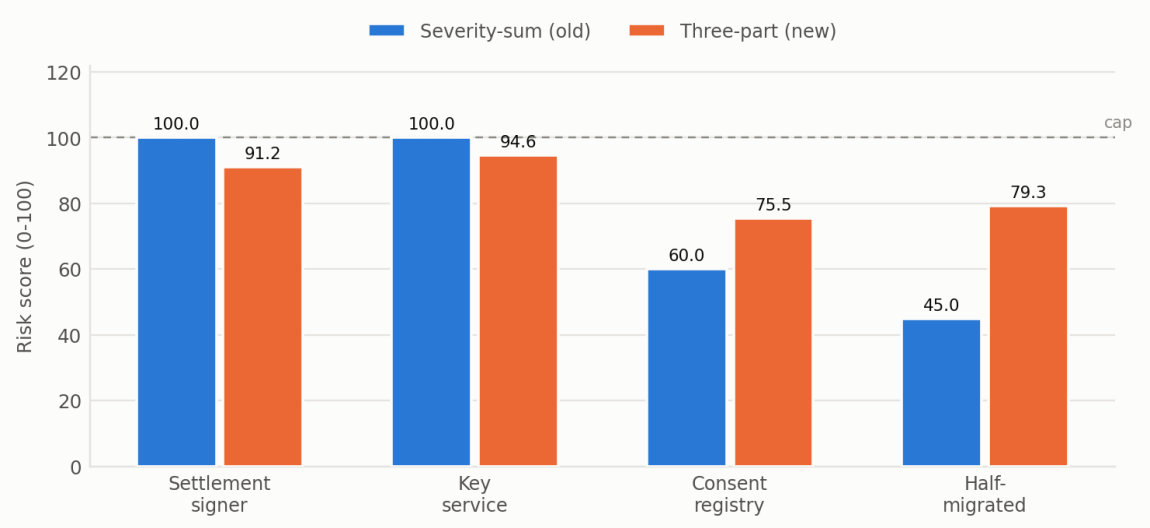


Fig. 3. Old and new exposure scores for the four affected files. The old model saturates at the cap for the two worst files; the new model assigns four distinct values.

The predicted saturation appears exactly where Section 3.6 anticipates it. The two files whose weighted severity sum exceeds the cap both receive 100.0, so the old model cannot distinguish a file with 9 findings across 108 lines from one with 5 findings across 26 lines, although the second is more than twice as densely affected. The 19 development scan records exhibit the same effect at greater scale, with a mean of 99.5 and almost no variance.

Two properties of the new model deserve statement rather than concealment. On small files it can return a higher value than the severity sum: the half-migrated contract rises from 45.0 to 79.3, because two live high-severity families within 32 lines is genuinely dense exposure, which a count-based sum understates. And the ordering is not by finding count, since the key service outranks the settlement signer on density, 19.2 findings per hundred lines against 8.3. This is the intended behaviour, but it means the score must be read as exposure concentration rather than as a proxy for finding count.

4.4 Comparison with Baseline Models

Two kinds of baseline are relevant. The first is the detector's own earlier configuration, v1-baseline, reported in Table 3: precision improves from 0.633 to 1.000, an absolute gain of 0.367, with recall held at 1.000 and F1 rising from 0.776 to 1.000. Because v1 is a real prior version retained verbatim, this comparison measures an actual historical improvement rather than a reconstruction.

The second baseline is the existing tooling a practitioner would otherwise reach for. Slither, Mythril and Manticore report zero findings of this class by construction, because cryptographic composition lies outside their detection scope; they are not weaker at the task so much as not addressed to it. CryptoGuard and CrySL do analyse cryptography but target misuse rules in Java and report nothing for a correct ECDSA implementation. The comparison in Table 1 is therefore one of capability coverage rather than of competing accuracy figures, and no prior tool provides a quantitative baseline on this specific task. Establishing one is part of the contribution of

releasing the benchmark.

What these numbers do and do not establish. The absolute scores must not be read as field accuracy. The benchmark is small, at seven files and nineteen call sites, and it was authored alongside the detector by the same author; under those conditions a perfect score for the final configuration is close to guaranteed by construction, because the hard negatives were written to exercise error classes already identified for repair. What the ablation does establish is narrower and still useful: each mechanism removes the error class it targets, the removal is complete even on data designed to be adversarial, and recall does not pay for it. The defensible reading is the change from 0.633 to 1.000 under a comparison that isolates its cause.

Sample size compounds this. Nineteen correct predictions out of nineteen support a Wilson 95% interval on precision of only [0.832, 1.000]. Per family the position is weaker still: the single short-key AES call site yields [0.207, 1.000], which is to say nothing at all about that family. No particular level of accuracy is therefore claimed from this benchmark, only a difference between configurations measured on identical data. An interval tight enough to support a claim above 0.95 would require on the order of 150 labelled call sites.

4.5 Runtime Equivalence

The equivalence harness reports agreement between the Python and JavaScript implementations on all seven benchmark files across all six compared fields. The browser runtime therefore carries the measured behaviour of the evaluated reference implementation rather than an approximation of it, which is a necessary condition for reporting results obtained from one implementation while deploying another.

5. Discussion

5.1 Answers to the Research Questions

RQ1. What causes false positives, and how much does each contribute? Two causes account for all eleven false positives observed in the baseline. Matches inside comments, documentation strings and string literals are the dominant cause, and duplicate matches on a single call site by overlapping patterns are the second. A third cause, substring matches inside non-cryptographic identifiers, is real and reproducible but contributed no false positive on this data, which is itself informative: the intuitive first remedy addresses the least damaging cause.

RQ2. Can those causes be eliminated without loss of recall? Yes. Offset-preserving masking and per-line grouping together remove all eleven false positives while recall remains exactly 1.000, and raw hits fall from 38 to 26. Preserving offsets rather than deleting characters is what makes this possible without sacrificing the line numbers that give the output its value.

RQ3. Does severity-sum scoring discriminate, and does a multi-factor score improve on it? Severity-sum scoring does not discriminate: it assigns the identical maximum to files differing by a factor of 4.5 in finding count and a factor of 2 in family breadth, and produced a mean of 99.5 across 19 development scans. The three-term model assigns four distinct values to the four affected files and preserves zero across all three negatives, so it discriminates where the previous model could not. It is important to be precise about the scope of this claim: discrimination is demonstrated, validity is not, since no study has established that the model's ordering matches expert risk judgement.

RQ4. Can the logic be ported to a client-side runtime without divergence? Yes, and the question is worth asking precisely because maintaining two implementations invites drift that would invalidate the evaluation. Agreement on all seven files across six fields is verified mechanically and is intended to be re-run whenever the pattern set changes.

5.2 Comparison with Previous Studies

The accuracy result sits in a different position from the figures reported by CryptoGuard [6], which achieves low false-positive rates on large Java codebases by means of refinement insights specific to misuse rules. That work reduces false positives arising from imprecise data flow; the false positives characterised here arise earlier, at the lexical layer, and are therefore orthogonal to it. A tool combining both would be stronger than either, and the mechanisms contributed here are independent of language and of the downstream analysis.

Relative to CrySL [7], which requires a usage specification per API, the approach here needs no specification and consequently gives up the ability to judge whether an algorithm is used correctly. The two address complementary

questions, and the limitation is intrinsic rather than incidental.

Against the smart contract analysis literature represented by Slither [5], there is no overlap in detected defect classes. That literature has treated cryptographic composition as out of scope because, until the standardisation of post-quantum algorithms, it was not actionable. Our position is that standardisation has changed this: with FIPS 203, 204 and 205 finalised and a transition timetable published [11], which algorithms a contract composes is now a reportable property with a defined remediation.

On the quantitative side, the resource estimates of Webber et al. [13] inform urgency rather than method, and this work takes no position on the timeline. The relevant deadline, as Mosca's framing [4] makes clear and as the transition guidance [11] formalises, is set by secrecy lifetime and migration duration rather than by any single hardware estimate, which is what makes an inventory tool useful now rather than later.

6. Conclusion and Future Work

6.1 Summary of Findings

Organisations operating blockchain systems must migrate their cryptography, and the first step is an inventory of where the vulnerable primitives are invoked. Existing tools do not produce it: smart contract analysers report logic defects, and cryptographic analysers report incorrect use of sound primitives, whereas the problem here is correct use of primitives that are about to become unsound. This paper presented a toolkit that produces that inventory and carries it through to migration planning, standards-conformant export, regulator-facing documentation and a verifiable record.

The substantive finding concerns accuracy. A pattern-based detector for this task is easy to make complete and hard to make precise, and its errors are dominated by two systematic classes rather than by scattered noise. Offset-preserving masking and per-line family grouping remove both, raising precision from 0.633 to 1.000 at unchanged recall on a benchmark constructed to be adversarial for this class of detector, while word-boundary anchoring alone removes no false positive at all. The severity-sum scoring such tools normally use was shown to saturate, assigning an identical maximum to files differing severalfold in extent, and a three-term alternative was given that separates them. The detection logic was additionally shown to port to a browser runtime with verified identical behaviour.

6.2 Limitations

- **Detection is lexical.** The tool identifies cryptographic API usage but cannot determine whether a correctly named primitive is used correctly, and cannot follow indirection: a project wrapping ECDSA behind a generic signing facade yields one finding at the wrapper rather than one per caller. For migration planning this is often desirable, since the wrapper is the correct place to change, but it understates the blast radius.
- **Masking approximates parsing.** Unusual or nested string delimiters can mask imprecisely. Damage is bounded by forbidding single-line literal patterns from crossing newlines, but a per-language parser would be strictly more accurate.
- **Construct validity is the dominant evaluation limitation.** The benchmark was authored alongside the detector, so the hard negatives test error classes already identified for repair and the final scores are correspondingly optimistic.
- **External validity is limited.** Seven files of a few dozen lines cannot represent production blockchain codebases, which are larger, use more libraries and contain idioms not anticipated here.
- **The matching criterion is strict.** A finding one line from its label counts as both a false positive and a false negative, penalising near misses more heavily than a developer would.
- **Deployment scope.** Blockchain records are written to a test network and so carry no economic finality, and certificates are backed by a local registry rather than a public verification service.

6.3 Future Scope

The clear priority is an independent evaluation corpus, and the harness for it has been implemented and released so that the protocol is fixed before any data is collected. It operates in three stages whose separation does the methodological work. The first samples contracts from creation transactions in randomly chosen recent blocks, retains those with verified source, and generates candidate lines using a search deliberately cruder than the

detector, so that the candidate set is a strict superset of anything the detector can report and detector false negatives become measurable. The second presents each candidate with surrounding context to a human annotator; it does not import the detector and never displays its prediction, because an annotator shown a prediction produces labels that inherit the detector's biases. Two annotators label independently and disagreements are excluded rather than resolved by majority vote. The third reports Cohen's kappa alongside precision, recall and F1 with Wilson intervals, treating kappa below approximately 0.6 as a gate requiring the labelling policy to be revised before any accuracy figure is quoted.

Three further directions follow. Replacing lexical masking with per-language parsing would eliminate the approximation noted above and would permit call-graph analysis, resolving indirection through wrappers and reporting the true blast radius. Calibrating the weights of Equation (1) against expert risk rankings would move the score from discriminating to validated. Finally, measuring the gas cost of on-chain lattice verification under current and proposed precompiles would establish when the off-chain architectural workaround can be retired in favour of direct verification.

7. References

[1] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM J. Comput.*, vol. 26, no. 5, pp. 1484–1509, 1997.

[2] L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *Proc. 28th Annu. ACM Symp. Theory of Computing (STOC)*, 1996, pp. 212–219.

[3] S. Nakamoto, “Bitcoin: a peer-to-peer electronic cash system,” White Paper, 2008.

[4] M. Mosca, “Cybersecurity in an era with quantum computers: will we be ready?,” *IEEE Security & Privacy*, vol. 16, no. 5, pp. 38–41, 2018.

[5] J. Feist, G. Grieco, and A. Groce, “Slither: a static analysis framework for smart contracts,” in *Proc. 2nd Int. Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, 2019, pp. 8–15.

[6] S. Rahaman *et al.*, “CryptoGuard: high precision detection of cryptographic vulnerabilities in massive-sized Java projects,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2019, pp. 2455–2472.

[7] S. Krüger, J. Späth, K. Ali, E. Bodden, and M. Mezini, “CrySL: an extensible approach to validating the correct usage of cryptographic APIs,” *IEEE Trans. Softw. Eng.*, vol. 47, no. 11, pp. 2382–2400, 2021.

[8] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard,” NIST FIPS 203, 2024.

[9] National Institute of Standards and Technology, “Module-lattice-based digital signature standard,” NIST FIPS 204, 2024.

[10] National Institute of Standards and Technology, “Stateless hash-based digital signature standard,” NIST FIPS 205, 2024.

[11] National Institute of Standards and Technology, “Transition to post-quantum cryptography standards,” NIST IR 8547, 2024.

[12] D. J. Bernstein and T. Lange, “Post-quantum cryptography,” *Nature*, vol. 549, no. 7671, pp. 188–194, 2017.

[13] M. Webber, V. Elfving, S. Weidt, and W. B. Hensinger, “The impact of hardware specifications on reaching quantum advantage in the fault tolerant regime,” *AVS Quantum Sci.*, vol. 4, no. 1, 013801, 2022.

[14] OWASP Foundation, “CycloneDX bill of materials specification, version 1.6,” 2024.

[15] E. Barker, “Recommendation for key management: Part 1 — general,” NIST SP 800-57 Part 1 Rev. 5, 2020.